# RANCANG BANGUN SISTEM TRANSKRIPSI AUDIO BERBASIS WEB DENGAN EVALUASI KOMPARATIF MODEL WHISPER PADA KORPUS PERCAKAPAN BAHASA INDONESIA

**PROPOSAL PENELITIAN**

UIN IMAM BONJOL
PADANG


**Oleh:**

**MUHAMMAD IFFAN RAMADHAN**

**NIM. 2117020036**



**PROGRAM STUDI SISTEM INFORMASI**
**FAKULTAS SAINS DAN TEKNOLOGI**
**UNIVERSITAS ISLAM NEGERI IMAM BONJOL PADANG**
**PADANG**


**2026**

# HALAMAN PERSETUJUAN

Proposal penelitian dengan judul **"Rancang Bangun Sistem Transkripsi Audio Berbasis Web dengan Evaluasi Komparatif Model Whisper pada Korpus Percakapan Bahasa Indonesia"** yang disusun oleh MUHAMMAD IFFAN RAMADHAN NIM 2117020036, telah diperiksa dan disetujui untuk diajukan dalam seminar proposal pada Program Studi Sistem Informasi Fakultas Sains dan Teknologi Universitas Islam Negeri Imam Bonjol Padang.

Padang, ..... 2026

Menyetujui:

| Pembimbing I | Pembimbing II |
|---|---|
| **Ahmad Fauzi, M.T.I.** | **Ozzy Secio Riza, M.Kom.** |
| NIP. 198805092018011001 | NIP. 199310122019032015 |

# KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia-Nya sehingga proposal penelitian yang berjudul "Rancang Bangun Sistem Transkripsi Audio Berbasis Web dengan Evaluasi Komparatif Model Whisper pada Korpus Percakapan Bahasa Indonesia" ini dapat diselesaikan. Shalawat serta salam semoga senantiasa tercurah kepada Nabi Muhammad SAW sebagai uswatun hasanah bagi umat manusia.

Proposal penelitian ini disusun sebagai salah satu syarat untuk melaksanakan penelitian skripsi pada Program Studi Sistem Informasi, Fakultas Sains dan Teknologi, Universitas Islam Negeri Imam Bonjol Padang. Penulis menyadari bahwa penyelesaian proposal ini tidak lepas dari bimbingan, dukungan, dan bantuan dari berbagai pihak. Oleh karena itu, penulis menyampaikan ucapan terima kasih kepada Bapak Ahmad Fauzi, M.T.I. selaku Pembimbing I dan Ibu Ozzy Secio Riza, M.Kom. selaku Pembimbing II yang telah memberikan arahan dan bimbingan yang sangat berharga.

Penulis menyadari bahwa proposal ini masih memiliki ruang untuk penyempurnaan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan pelaksanaan penelitian ke depan. Semoga proposal penelitian ini dapat memberikan kontribusi nyata bagi pengembangan ilmu pengetahuan dan penerapan teknologi informasi.

Padang, . . . . . 2026

Penulis

## DAFTAR ISI


**HALAMAN PERSETUJUAN** ..... **ii**

**KATA PENGANTAR** ..... **iii**

**DAFTAR ISI** ..... **iv**

**DAFTAR TABEL** ..... **v**

**DAFTAR GAMBAR** ..... **vi**

**BAB I PENDAHULUAN** ..... **1**

A Latar Belakang ..... 1

B Rumusan Masalah ..... 5

C Batasan Masalah ..... 5

D Tujuan Penelitian ..... 5

E Manfaat Penelitian ..... 6

**BAB II TINJAUAN PUSTAKA** ..... **8**

A Kerangka Teori ..... 8

B Literatur Review ..... 21

**BAB III METODE PENELITIAN** ..... **27**

A Jenis Penelitian ..... 27

B Setting Penelitian ..... 27

C Prosedur Penelitian ..... 28

D Data dan Sumber Data ..... 30

E Alat dan Instrumen Penelitian ..... 31

F Teknik Analisis Data ..... 32

G Jadwal Penelitian ..... 34

**BAB IV PENUTUP** ..... **35**

A Kesimpulan ..... 35

B Saran ..... 35

**DAFTAR PUSTAKA** ..... **36**

# DAFTAR TABEL

2.1 Simbol, Nama, dan Fungsi Notasi UML..... 17
2.2 Ringkasan Literatur Review ..... 22
3.1 Statistik Dataset Penelitian..... 31
3.2 Jadwal Penelitian (*Gantt Chart*) ..... 34

## DAFTAR GAMBAR

3.1 Alur Penelitian (*Flowchart*)..... 30

# BAB I
# PENDAHULUAN

## A. Latar Belakang

Pendokumentasian informasi dari rekaman audio, seperti rekaman siniar, wawancara penelitian, dan persidangan akademik, membutuhkan proses transkripsi teks sebagai langkah awal pengolahan data. Keberadaan teks transkripsi mempermudah pengindeksan, pencarian kata kunci, serta pengarsipan dokumen digital. Kendati demikian, proses transkripsi manual membutuhkan waktu dan tenaga yang besar. Konversi audio secara manual dapat memerlukan waktu lebih dari lima kali durasi rekaman yang ditranskripsikan (Gaur dkk., 2016).

Beban tersebut tidak hanya muncul pada tahap pengetikan, tetapi juga pada proses mendengarkan ulang, menentukan batas segmen, menyesuaikan tanda baca, dan memeriksa istilah yang tidak terdengar jelas. Pada rekaman percakapan, pekerjaan transkripsi menjadi lebih kompleks karena penutur dapat berbicara dengan kecepatan berbeda, menggunakan ungkapan tidak baku, atau menghasilkan ucapan yang saling bertumpuk. Dalam kondisi demikian, transkriptor harus berulang kali berpindah antara pemutar audio dan dokumen teks. Proses tersebut meningkatkan waktu penyelesaian dan membuka peluang terjadinya ketidakkonsistenan penulisan antarkalimat.

Pemanfaatan keluaran ASR sebagai draf dapat mengurangi sebagian pekerjaan tersebut, tetapi manfaatnya bergantung pada kualitas hasil pengenalan. Keluaran ASR yang memiliki tingkat kesalahan tinggi dapat membutuhkan waktu koreksi lebih lama daripada transkripsi dari awal. Temuan ini menunjukkan bahwa sistem transkripsi tidak cukup hanya menghasilkan teks, tetapi juga perlu menyediakan informasi yang membantu pengguna menilai, menelusuri, dan memperbaiki kesalahan secara cepat (Gaur dkk., 2016).

Perkembangan model *Automatic Speech Recognition* (ASR) modern berbasis arsitektur *Transformer*, seperti model OpenAI *Whisper*, memberikan peluang untuk mengotomatisasi konversi suara ke teks. Model tersebut dilatih menggunakan

680.000 jam data audio multibahasa dan menunjukkan kemampuan generalisasi pada berbagai tolok ukur pengenalan suara (Radford dkk., 2022). Namun, penerapan ASR pada rekaman percakapan alami berbahasa Indonesia menghadapi tantangan tersendiri. Percakapan spontan dapat memuat variasi dialek, alih kode (*code-switching*), bahasa percakapan nonformal, dan tumpang tindih suara antarpembicara; karakteristik tersebut dapat menurunkan akurasi transkripsi. Oleh karena itu, hasil ASR perlu diperlakukan sebagai draf yang masih harus diverifikasi dan disunting oleh manusia. Layanan transkripsi berbasis *web* yang mengintegrasikan pemrosesan suara dengan penyuntingan transkrip dan anotasi pembicara telah dikembangkan, sehingga alur koreksi dapat menjadi bagian dari sistem transkripsi (Olev & Alumäe, 2022).

Kinerja ASR juga dipengaruhi oleh karakteristik bahan uji. Sistem yang bekerja baik pada ucapan terarah belum tentu memberikan hasil yang sama pada siniar atau percakapan spontan. Kebisingan, karakteristik penutur, dan tumpang tindih suara dapat memengaruhi kesalahan transkripsi pada beragam kondisi percakapan (Imai dkk., 2025). Pada konteks siniar, pola kesalahan juga dapat berbeda menurut jenis disfluensi, seperti interjeksi atau ucapan yang telah disunting (Teleki dkk., 2024). Karena itu, evaluasi perlu menggunakan data percakapan yang mendekati kondisi penggunaan sebenarnya, bukan hanya data dengan artikulasi yang bersih.

Masalah lain terletak pada pengolahan audio berdurasi panjang. Siniar dalam penelitian ini disimpan sebagai berkas utuh per episode, sedangkan sistem ASR dapat membagi audio menjadi beberapa segmen untuk diproses. Batas segmen yang kurang tepat dapat memengaruhi konteks ujaran dan ketepatan tanda waktu. WhisperX menggabungkan deteksi aktivitas suara, inferensi berkelompok, dan *forced alignment* untuk menghasilkan tanda waktu pada tingkat kata (Bain dkk., 2023). Dengan demikian, akurasi transkripsi dan ketepatan waktu keluaran perlu dipertimbangkan secara bersamaan ketika sistem digunakan untuk menyunting rekaman panjang.

Dalam penggunaan berbahasa Indonesia, pemilihan model perlu mempertimbangkan karakteristik bahasa dan sumber daya komputasi. Ukuran

model dan konfigurasi inferensi dapat memengaruhi kebutuhan komputasi serta kualitas transkripsi. Penelitian tentang optimisasi Whisper untuk bahasa Indonesia, misalnya, mengkaji penggunaan pendekatan *hybrid cloud* dan beberapa mesin pemrosesan untuk menyeimbangkan kinerja sistem (Fathony dkk., 2025). Karena itu, kapasitas perangkat keras dan biaya komputasi perlu diperhitungkan saat menentukan konfigurasi sistem.

Tantangan berikutnya berkaitan dengan infrastruktur dan tata kelola data. Layanan ASR komersial dan perangkat lunak sumber terbuka memiliki perbedaan dalam kualitas, kecepatan, dan kendali atas lingkungan pemrosesan. Pengiriman audio ke layanan pihak ketiga juga perlu dipertimbangkan berdasarkan kebijakan dan kebutuhan perlindungan data lembaga. Sementara itu, pemrosesan model berukuran besar pada perangkat keras lokal memerlukan sumber daya komputasi yang memadai. Oleh sebab itu, pemisahan aplikasi *web* dari unit inferensi dapat membantu mengelola pekerjaan transkripsi yang membutuhkan komputasi intensif. Kajian komparatif layanan ASR menunjukkan bahwa kinerja dapat berubah menurut karakteristik masukan, sehingga pemilihan sistem perlu mempertimbangkan jenis audio yang akan diproses (Ferraro dkk., 2023).

Perbandingan antara layanan komersial dan perangkat lunak sumber terbuka menunjukkan adanya pertukaran antara kualitas, biaya, kendali data, dan kemudahan pengelolaan. Layanan komersial menyediakan infrastruktur siap pakai, sedangkan sistem sumber terbuka memberi ruang untuk mengatur lingkungan pemrosesan sesuai kebutuhan, tetapi memerlukan pengelolaan perangkat keras dan layanan pendukung. Perbedaan kinerja berdasarkan jenis masukan juga menunjukkan pentingnya evaluasi menggunakan kondisi audio yang relevan dengan penggunaan sistem (Ferraro dkk., 2023).

Berdasarkan pertimbangan tersebut, sistem yang dibangun perlu memisahkan pekerjaan yang bersifat interaktif dari pekerjaan inferensi yang membutuhkan komputasi tinggi. Pengunggahan berkas, penampilan status, dan penyuntingan teks harus tetap responsif meskipun unit pemrosesan sedang menangani audio berdurasi panjang. Pemisahan ini juga memungkinkan pengujian beberapa varian model secara

lebih teratur karena setiap pekerjaan dapat dicatat berdasarkan model, durasi audio, waktu inferensi, penggunaan VRAM, dan hasil transkripsinya. Dengan pencatatan tersebut, keputusan pemilihan model tidak hanya didasarkan pada ukuran parameter, tetapi juga pada kesesuaian antara akurasi, kecepatan, dan kebutuhan sumber daya.

Berdasarkan permasalahan tersebut, penelitian ini mengkaji sistem transkripsi audio berbasis *web* bernama Skriptor. Sistem memisahkan aplikasi berbasis *Next.js* dari unit pemrosesan inferensi berbasis *Python Worker*. Komunikasi asinkron menggunakan antrean tugas *Redis BullMQ*; progres transkripsi disampaikan kepada pengguna, sedangkan hasil dapat diperiksa dan dikoreksi melalui antarmuka yang menyelaraskan segmen teks dengan audio. Penelitian ini berfokus pada evaluasi komparatif tujuh varian model *Whisper* pada korpus percakapan bahasa Indonesia dan integrasi alur transkripsi tersebut ke dalam sistem *web*. Aplikasi menyediakan pilihan model operasional *small*, *turbo*, dan *large*; pilihan ini berbeda dari cakupan model pada eksperimen komparatif.

Selain menghasilkan teks, sistem ini dirancang untuk mendukung proses pemeriksaan oleh pengguna. Hasil transkripsi perlu disajikan bersama informasi waktu dan bagian audio yang terkait agar kesalahan dapat ditelusuri kembali ke sumbernya. Antarmuka yang demikian menempatkan model ASR sebagai alat bantu, sedangkan keputusan akhir mengenai ketepatan isi tetap berada pada pengguna. Pendekatan tersebut sesuai dengan karakteristik data percakapan yang sering memuat nama diri, istilah lokal, dan konteks yang tidak dapat ditentukan hanya dari sinyal suara.

Dengan mempertimbangkan kebutuhan tersebut, penelitian ini mengukur akurasi menggunakan WER dan CER, efisiensi komputasi menggunakan RTF dan penggunaan VRAM, serta persepsi kegunaan antarmuka menggunakan SUS setelah responden menyelesaikan skenario tugas. Kontribusi penelitian diarahkan pada penyajian bukti komparatif kinerja tujuh varian *Whisper* pada korpus percakapan bahasa Indonesia serta dokumentasi integrasinya dalam alur kerja aplikasi transkripsi berbasis *web*. Inferensi untuk evaluasi dilakukan pada GPU *Runpod*; penerapan lokal merupakan kemungkinan *deployment* apabila perangkat pengguna memenuhi

kebutuhan komputasi, bukan kondisi eksperimen yang diuji dalam penelitian ini.

## B. Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam penelitian ini dirumuskan sebagai berikut:

1. Bagaimana penerapan model RAD dalam penyempurnaan dan pengujian prototipe sistem transkripsi audio berbasis *web* Skriptor?
2. Bagaimana perbandingan akurasi (WER dan CER), waktu inferensi, RTF, dan penggunaan VRAM tujuh varian model *Whisper* pada seluruh data percakapan berbahasa Indonesia yang digunakan dalam penelitian?
3. Bagaimana hasil uji tugas dan skor SUS dalam menggambarkan kegunaan antarmuka transkripsi dan koreksi pada sistem Skriptor?

## C. Batasan Masalah

Penelitian ini dibatasi pada ruang lingkup sebagai berikut:

1. Eksperimen komparatif mencakup tujuh varian *Whisper*: *Tiny, Base, Small, Medium, Large-v2, Large-v3,* dan *Turbo*. Varian yang diuji dalam eksperimen tidak seluruhnya tersedia sebagai pilihan model pada antarmuka aplikasi.
2. Sistem dibangun menggunakan kerangka kerja *Next.js* 16 dengan bahasa *TypeScript* pada antarmuka *web*, serta unit *worker* berbasis *Python* yang terhubung melalui antrean terdistribusi berbasis *Redis BullMQ*.
3. Data pengujian menggunakan korpus percakapan bahasa Indonesia dari *Mozilla Data Collective* (MDC) yang terdiri dari 58 berkas audio utuh per episode siniar (*Homostoria* dan *Hari Minggoean*) dengan total durasi 23,5 jam, yang dievaluasi terhadap 11.540 segmen transkrip pada berkas acuan (*ground truth*) bertaraf TSV.
4. Seluruh pengujian inferensi komparatif dilakukan pada GPU NVIDIA RTX A5000 dengan VRAM 24 GB melalui layanan *Runpod*. Pengujian tidak membandingkan kinerja inferensi pada perangkat lokal.

## D. Tujuan Penelitian

Tujuan yang hendak dicapai dalam penelitian ini adalah:

1. Mendokumentasikan dan menguji prototipe sistem transkripsi audio berbasis

*web* Skriptordengan kerangka RAD dan arsitektur yang memisahkan aplikasi (*Next.js*) dari unit inferensi (*Python Worker*).

2. Mengevaluasi performa kecepatan inferensi, efisiensi konsumsi memori VRAM, dan tingkat akurasi transkripsi (WER dan CER) dari berbagai varian model OpenAI *Whisper* pada data rekaman percakapan bahasa Indonesia.
3. Mengevaluasi kegunaan antarmuka melalui penyelesaian skenario tugas dan pengukuran SUS terhadap 30 responden.

## E. Manfaat Penelitian

Manfaat yang diharapkan dari hasil penelitian ini meliputi: Secara teoretis, penelitian ini menyajikan data empiris mengenai performa tujuh varian model ASR *Whisper* pada rekaman percakapan alami berbahasa Indonesia serta penerapan arsitektur pemrosesan terdistribusi pada sistem transkripsi berbasis *web*. Evaluasi antarmuka melengkapi analisis melalui uji tugas dan pengukuran SUS; penelitian ini tidak mengklaim menerapkan seluruh siklus UCD.

Secara praktis, penelitian ini mengevaluasi prototipe sistem transkripsi bernama Skriptor yang memisahkan aplikasi *web* dari unit inferensi. Sistem dapat diterapkan pada lingkungan lokal apabila perangkat keras dan konfigurasi pendukungnya memadai. Namun, karena eksperimen penelitian menggunakan GPU melalui layanan *Runpod*, penelitian ini tidak mengukur penghematan biaya atau menjamin kerahasiaan data pada *deployment* lokal.

– Active: 1788614897217@@mirx-server.lan@5432@skriptor

# BAB II
# TINJAUAN PUSTAKA

## A. Kerangka Teori

Bagian ini memaparkan teori-teori dasar yang menjadi landasan pengembangan dan analisis sistem transkripsi terdistribusi. Alur tinjauan dimulai dari konsep *Automatic Speech Recognition* (ASR) dan arsitektur model *Whisper* sebagai mesin transkripsi utama, dilanjutkan dengan karakteristik bahasa Indonesia yang menjadi konteks penelitian, kemudian *pipeline WhisperX* dan teknik optimasi inferensi yang digunakan, hingga metrik evaluasi, arsitektur sistem terdistribusi, aspek *usability*, dan pengujian perangkat lunak. Teori-teori ini menjadi dasar untuk merancang dan mengevaluasi sistem pada Bab 3.

### 1. Automatic Speech Recognition (ASR)

*Automatic Speech Recognition* (ASR) adalah teknologi yang memungkinkan komputer untuk mengubah sinyal suara menjadi teks secara otomatis. Proses ini melibatkan konversi gelombang akustik mentah menjadi representasi digital yang kemudian dipetakan ke dalam satuan bahasa (fonem, suku kata, atau kata). Secara tradisional, sistem ASR mengandalkan komponen terpisah seperti model akustik, model bahasa, dan leksikon pelafalan. Namun, perkembangan terkini telah bergeser ke arah model *End-to-End* (E2E) yang menyederhanakan alur kerja tersebut menjadi satu jaringan saraf tunggal yang melakukan pemetaan langsung dari audio ke teks (Prabhavalkar dkk., 2023).

### 2. Arsitektur Model Whisper (OpenAI)

Model *Whisper* yang dikembangkan oleh OpenAI merupakan arsitektur jaringan saraf *End-to-End* (E2E) berbasis *Transformer Encoder-Decoder* yang dilatih menggunakan pendekatan *weakly supervised learning* pada skala data sebesar 680.000 jam audio multibahasa (Radford dkk., 2022). Arsitektur ini terdiri dari tiga komponen utama: pra-pemrosesan audio, *encoder*, dan *decoder* autoregresif.

Pada tahap pra-pemrosesan audio (*Audio Pre-processing*), sinyal audio mentah (*raw audio*) diubah menjadi berfrekuensi tunggal (*monophonic*) dengan laju sampel (*sampling rate*) 16.000 Hz. Sinyal tersebut kemudian dipecah menjadi potongan (*chunks*) berdurasi 30 detik. Ekstraksi fitur dilakukan menggunakan transformasi Fourier jangka pendek (*Short-Time Fourier Transform* atau STFT) untuk menghasilkan spektrogram log-Mel berukuran 80 dimensi, atau 128 dimensi pada varian *Whisper v3*.

*Encoder* Transformer memetakan spektrogram masukan menjadi representasi fitur akustik. Dimensi waktu potongan audio terlebih dahulu direduksi melalui blok konvolusi dan dilengkapi *positional embeddings*. Representasi tersebut kemudian diproses oleh tumpukan blok *Transformer Encoder* yang terdiri atas mekanisme *Multi-Head Self-Attention* (MHA) dan jaringan *Feed-Forward* (FFN) dengan normalisasi lapisan *Pre-LN*.

*Decoder* Transformer berfungsi sebagai model bahasa kondisional (*conditional language model*) yang memprediksi urutan token secara autoregresif berdasarkan representasi akustik dari *encoder*. Mekanisme *masked Multi-Head Self-Attention* mencegah model melihat token berikutnya, sedangkan *Multi-Head Cross-Attention* menghubungkan *decoder* dengan representasi akustik *encoder*.

Selain itu, model *Whisper* mengintegrasikan beberapa tugas pemrosesan suara melalui token kontrol khusus (*special tokens*) pada awal urutan *decoder*. Token tersebut mencakup penanda awal transkripsi, bahasa sasaran seperti $\langle|\text{id}|\rangle$, jenis tugas berupa *transcribe* atau *translate*, serta token kontrol tanda waktu.

### 3. Karakteristik Bahasa Indonesia untuk ASR

Bahasa Indonesia memiliki tantangan tersendiri dalam ASR karena variasi dialek dan statusnya yang sering dikategorikan sebagai *low-resource language* dalam beberapa korpus besar. Meskipun Whisper memiliki performa multibahasa yang baik, akurasinya pada bahasa Indonesia sering kali lebih rendah dibandingkan bahasa Inggris, sehingga memerlukan evaluasi spesifik menggunakan *dataset* lokal (Fathony dkk., 2025). Selain itu, percakapan alami berbahasa Indonesia, seperti pada

siniar, kerap mengandung bahasa gaul, ambiguitas linguistik, dan gangguan latar belakang yang menambah tingkat kesulitan transkripsi. Karakteristik ini menjadi dasar pemilihan korpus siniar berbahasa Indonesia sebagai objek pengujian dalam penelitian ini.

### 4. Inovasi *Pipeline* WhisperX

Meskipun *Whisper* memiliki kemampuan generalisasi yang sangat baik, model ini menghadapi keterbatasan serius ketika diterapkan pada audio berdurasi panjang, seperti masalah pergeseran tanda waktu (*timestamp drift*), halusinasi teks pada area hening (*hallucination loops*), dan latensi komputasi yang tinggi akibat sifat pemrosesan sekuensial autoregresif. Untuk mengatasi masalah tersebut, Bain dkk. (2023) memperkenalkan *WhisperX*, sebuah kerangka kerja terpadu berkinerja tinggi yang menggabungkan deteksi aktivitas suara, inferensi berkelompok, dan penyelarasan fonem paksa.

Sebelum audio dikirim ke model ASR, *WhisperX* menerapkan *Voice Activity Detection* (VAD) dengan strategi *Cut & Merge*. VAD memisahkan segmen bicara aktif dari bagian hening, kemudian menggabungkan segmen aktif secara dinamis hingga panjang maksimal 30 detik. Dengan demikian, pemotongan berlangsung pada fase hening alami dan kecenderungan *Whisper* menghasilkan pengulangan frasa (*repetition loops*) dapat dikurangi.

Berbeda dari pemrosesan berurutan (*sequential decoding*) pada *Whisper* asli, *batched transcription* melalui *Faster-Whisper* memungkinkan paralelisasi komputasi pada GPU. Karena pembagian segmen berbasis VAD menghasilkan batas potongan yang bersih, setiap segmen dapat ditranskripsikan secara mandiri dan paralel dalam satu kelompok (*batch*). *WhisperX* mengintegrasikan *Faster-Whisper*, yaitu implementasi model *Whisper* dalam *C++* yang menggunakan pustaka optimasi *CTranslate2*. Kombinasi *batching* dan optimasi tersebut mempercepat waktu pemrosesan pada GPU NVIDIA modern.

Penanda waktu bawaan *Whisper* diprediksi pada tingkat frasa dengan presisi kasar dan rentan terhadap pergeseran temporal (*drifting*). Untuk memperoleh

penanda waktu tingkat kata (*word-level timestamp*) yang akurat, *WhisperX* menerapkan *forced alignment* menggunakan model berbasis *Connectionist Temporal Classification* (CTC), seperti *Wav2Vec 2.0* atau *WavLM*. Hasil pemetaan ini memberikan koordinat batas waktu awal dan akhir ($t_{start}, t_{end}$) setiap kata.

Tahap terakhir dalam *pipeline WhisperX* adalah *speaker diarization*, yaitu klasifikasi identitas suara untuk menentukan siapa yang berbicara pada setiap kata (*who spoke when*). Pustaka *PyAnnote Audio* mengekstraksi *speaker embeddings*, misalnya menggunakan *ECAPA-TDNN*, kemudian mengelompokkan pembicara dan memetakan identitasnya ke setiap kata berdasarkan koordinat waktu dari tahap *forced alignment*.

### 5. Optimasi Inferensi: Faster-Whisper dan Kuantisasi

Untuk menjalankan model besar pada lingkungan produksi yang efisien, diperlukan teknik optimasi inferensi yang mampu memangkas waktu pemrosesan dan konsumsi memori tanpa mendegradasi akurasi secara signifikan. Pustaka *CTranslate2* merupakan mesin inferensi berkinerja tinggi untuk model berarsitektur *Transformer*. Pustaka ini menerapkan peleburan operator (*operator fusion*), alokasi memori dinamis, serta paralelisasi CPU dan GPU untuk mengurangi *overhead* eksekusi *PyTorch*.

Kuantisasi model FP16 dan INT8 mengurangi presisi numerik bobot (*weights*) jaringan saraf. Pada inferensi FP32, setiap bobot membutuhkan 32 bit (4 bita), sedangkan INT8 merepresentasikannya dengan 8 bit (1 bita) melalui rumus:

$$W_q = \text{round}\left(\frac{W}{S} + Z\right) \tag{2.1}$$

di mana $W$ melambangkan bobot berpresisi pecahan kontinu, $S$ adalah faktor skala (*scale factor*), dan $Z$ merupakan titik nol (*zero-point*). Kuantisasi ini menghemat kapasitas memori VRAM GPU hingga 50–75% serta mempercepat perpindahan data pada memori (Andreyev, 2025). Kecepatan inferensi diukur menggunakan *Real-Time Factor* (RTF), yaitu perbandingan waktu komputasi model ($\Delta t_{\text{inferensi}}$)

terhadap durasi berkas audio ($\Delta t_{\text{audio}}$):

$$\text{RTF} = \frac{\Delta t_{\text{inferensi}}}{\Delta t_{\text{audio}}} \tag{2.2}$$

Nilai RTF $< 1{,}0$ menandakan proses inferensi berjalan lebih cepat dibandingkan durasi pemutaran audio di dunia nyata (*faster than real-time*). Kebalikan dari RTF dinyatakan sebagai faktor percepatan komputasi (*Speed Multiplier*):

$$\text{Speed} = \frac{1}{\text{RTF}} = \frac{\Delta t_{\text{audio}}}{\Delta t_{\text{inferensi}}} \tag{2.3}$$

### 6. Metrik Evaluasi Akurasi: WER dan CER

Akurasi sistem ASR dievaluasi secara kuantitatif berdasarkan jarak suntingan Levenshtein (*Levenshtein Distance*) yang mengukur perbedaan jumlah operasi minimum antara urutan token keluaran hipotesis model dengan urutan token berkas acuan manual (*ground truth*). *Word Error Rate* (WER) merupakan metrik utama untuk mengukur tingkat kesalahan pengenalan wicara pada tingkat kata. Nilai WER dihitung melalui persamaan:

$$\text{WER} = \frac{S+D+I}{N} = \frac{S+D+I}{S+D+C} \tag{2.4}$$

di mana:

- $S$ (*Substitutions*) adalah jumlah kata dalam referensi yang digantikan oleh kata lain pada keluaran model.
- $D$ (*Deletions*) adalah jumlah kata dalam referensi yang hilang atau tidak dikenali oleh model.
- $I$ (*Insertions*) adalah jumlah kata tambahan pada keluaran model yang sebenarnya tidak ada pada referensi.
- $C$ (*Correct*) adalah jumlah kata yang berhasil dikenali secara tepat dan identik.
- $N$ adalah total jumlah seluruh kata yang ada pada teks referensi acuan ($N = S+D+C$).

*Character Error Rate* (CER) merupakan metrik evaluasi kesalahan pada tingkat karakter yang dihitung dengan prinsip yang sama seperti WER:

$$\text{CER} = \frac{S_c + D_c + I_c}{N_c} \tag{2.5}$$

di mana $S_c$, $D_c$, $I_c$, dan $N_c$ dihitung pada satuan karakter individual. CER sangat esensial dalam analisis bahasa Indonesia untuk mendeteksi ketepatan imbuhan (afiksasi) dan partikel yang mungkin hanya menimbulkan sedikit variasi karakter namun mengubah makna kata secara keseluruhan (Ferraro dkk., 2023).

### 7. Arsitektur Sistem Terdistribusi

Sistem berskala besar yang menangani beban inferensi AI intensif memerlukan pemisahan tanggung jawab (*separation of concerns*) melalui arsitektur terdistribusi: Arsitektur *Message Queue Pattern* menempatkan peladen web sebagai produsen (*producer*) yang memasukkan pekerjaan ke dalam antrean terisolasi. Pustaka *BullMQ* berbasis *Redis* mengelola siklus hidup pekerjaan, mekanisme pengulangan saat gagal (*retry mechanism* dengan *exponential backoff*), serta penguncian tugas (*distributed locks*) untuk mencegah tabrakan eksekusi antar-pekerja.

Pada *Worker Pattern*, satu atau beberapa unit pekerja (*worker*) berjalan di lingkungan terpisah dan berfokus pada komputasi GPU. Pemisahan ini memungkinkan isolasi konkurensi, misalnya satu antrean untuk setiap GPU, sehingga penggunaan VRAM lebih stabil dan risiko galat kehabisan memori (*Out-of-Memory*) dapat dikurangi (Macháček dkk., 2023).

Progres inferensi disiarkan ke antarmuka klien melalui *Server-Sent Events* (SSE) yang dipicu oleh *Redis Pub/Sub*. Integritas data dari unit pemrosesan eksternal ke peladen web dijaga dengan verifikasi tanda tangan kriptografis menggunakan *Hash-based Message Authentication Code* (HMAC-SHA256).

### 8. Rapid Application Development (RAD)

*Rapid Application Development* (RAD) merupakan model pengembangan perangkat lunak yang menekankan pembuatan prototipe, konstruksi cepat, dan umpan balik berulang. Martin (1991) membagi RAD ke dalam empat fase: perencanaan kebutuhan (*requirements planning*), desain pengguna (*user design*), konstruksi (*construction*), serta penerapan (*cutover*). Fase-fase tersebut digunakan sebagai kerangka untuk mengatur kebutuhan, rancangan, pembangunan atau penyempurnaan prototipe, dan pengujian sebelum sistem digunakan.

RAD sesuai untuk pengembangan sistem yang kebutuhan dan antarmukanya dapat diperiksa melalui prototipe, tetapi penerapannya tetap memerlukan ruang lingkup dan jadwal yang terkendali. Dalam penelitian ini, prototipe awal Skriptor telah tersedia. Oleh karena itu, tahapan RAD didokumentasikan berdasarkan artefak pengembangan yang tersedia dan digunakan untuk mengatur penyempurnaan serta pengujian prototipe selama penelitian; tahapan atau keterlibatan pengguna yang tidak memiliki bukti tidak dinyatakan telah dilakukan.

### 9. Human-in-the-Loop (HITL), *Usability*, dan *System Usability Scale*

Konsep *Human-in-the-Loop* (HITL) mengakui bahwa sistem kecerdasan buatan, secanggih apa pun, tidak dapat sepenuhnya menggantikan penilaian manusia dalam tugas-tugas yang memerlukan pemahaman kontekstual dan ketelitian tinggi. Dalam konteks transkripsi, HITL menempatkan manusia sebagai validator akhir yang bertugas memeriksa dan mengoreksi hasil keluaran model ASR sebelum draf transkripsi dianggap final. Penelitian terdahulu menunjukkan bahwa alur kerja semi-otomatis yang menggabungkan draf transkripsi dari AI dengan antarmuka penyuntingan yang intuitif dapat meningkatkan produktivitas secara signifikan, terutama jika antarmuka tersebut dilengkapi dengan fitur visualisasi gelombang audio yang tersinkronisasi dengan teks (Olev & Alumäe, 2022; Imai dkk., 2025). Namun, efektivitas pendekatan HITL sangat bergantung pada tingkat kegunaan (*usability*) antarmuka yang disediakan bagi pengguna.

Antarmuka koreksi menjadi titik temu utama antara manusia dan sistem sehingga kegunaannya perlu dievaluasi dalam konteks tugas transkripsi. UCD secara umum mencakup pemahaman konteks penggunaan, perancangan, dan evaluasi yang melibatkan pengguna secara berulang (Riebel, 2023). Penelitian ini tidak menerapkan atau mengklaim seluruh siklus UCD. Keterlibatan pengguna dibatasi pada penyelesaian skenario tugas dan pengisian SUS untuk mengevaluasi prototipe yang telah tersedia.

Menurut standar internasional ISO 9241-11, *usability* didefinisikan sebagai sejauh mana suatu produk dapat digunakan oleh pengguna tertentu untuk mencapai tujuan tertentu dengan efektivitas (*effectiveness*), efisiensi (*efficiency*), dan kepuasan (*satisfaction*) dalam konteks penggunaan tertentu. Untuk mengukur aspek *usability* secara kuantitatif pada sistem yang dikembangkan, diperlukan instrumen pengukuran yang terstandarisasi dan reliabel.

*System Usability Scale* (SUS) yang dikembangkan oleh Brooke (1986) merupakan kuesioner terstandarisasi yang terdiri dari 10 butir pernyataan dengan skala Likert 5 poin (1 = Sangat Tidak Setuju hingga 5 = Sangat Setuju). Kuesioner ini dirancang untuk memberikan penilaian global terhadap kegunaan suatu sistem secara cepat dan murah. Sepuluh butir pernyataan SUS disusun secara bergantian antara pernyataan bernada positif (butir ganjil: 1, 3, 5, 7, 9) dan pernyataan bernada negatif (butir genap: 2, 4, 6, 8, 10). Prosedur perhitungan skor SUS adalah sebagai berikut:

1. Untuk setiap butir bernada positif (ganjil), kontribusi skor dihitung dengan mengurangi 1 dari respons pengguna: $(x_i - 1)$.
2. Untuk setiap butir bernada negatif (genap), kontribusi skor dihitung dengan mengurangi respons pengguna dari 5: $(5 - x_i)$.
3. Jumlahkan seluruh kontribusi skor dari 10 butir, lalu kalikan dengan 2,5 untuk mendapatkan skor akhir dalam rentang 0–100.

Secara matematis, rumus skor SUS dapat dituliskan sebagai:

$$\text{SUS} = 2{,}5 \times \left[ \sum_{i \in \text{ganjil}} (x_i - 1) + \sum_{j \in \text{genap}} (5 - x_j) \right] \tag{2.6}$$

Berdasarkan studi empiris yang dilakukan oleh Bangor dkk. (2008) terhadap lebih dari 2.000 survei, interpretasi skor SUS dikategorikan ke dalam tiga tingkatan penerimaan: skor di atas 70 dikategorikan sebagai *Acceptable* (dapat diterima), skor antara 50–70 dikategorikan sebagai *Marginal* (perlu perbaikan), dan skor di bawah 50 dikategorikan sebagai *Not Acceptable* (tidak dapat diterima). Selain itu, Bangor dkk. juga memetakan skor SUS ke dalam skala penilaian deskriptif (*adjective rating*): *Excellent* ($\geq$ 85,5), *Good* (72,6–85,5), *OK* (50,9–72,5), *Poor* (25,1–50,9), dan *Worst Imaginable* ($\leq$ 25,0).

## 10. Pengujian Perangkat Lunak

Pengujian perangkat lunak (*software testing*) merupakan proses evaluasi terhadap suatu sistem untuk memastikan bahwa perilaku dan kinerjanya telah sesuai dengan spesifikasi yang diharapkan serta terbebas dari kesalahan (*bugs* atau *errors*). Pengujian ini bertujuan untuk meningkatkan keandalan, keamanan, dan kualitas perangkat lunak secara keseluruhan sebelum dirilis ke lingkungan produksi. Secara garis besar, pengujian perangkat lunak dibagi menjadi beberapa metode dan tingkatan, antara lain: *Black-box testing* mengevaluasi fungsionalitas sistem tanpa memerlukan pengetahuan mengenai struktur kode atau logika internalnya. Pengujian dilakukan dengan memberikan *input* tertentu dan memverifikasi kesesuaiannya dengan *output* yang diharapkan.

*Unit testing* menguji komponen perangkat lunak terkecil, seperti fungsi, kelas, atau modul, secara terisolasi. Sementara itu, *integration testing* memverifikasi interaksi dan aliran data di antara modul yang saling terhubung. Pengujian otomatis (*automated testing*) menjalankan skenario yang telah diprogram menggunakan perangkat lunak khusus. Pendekatan ini mempercepat verifikasi dan membantu mencegah regresi (*regression*) ketika kode diperbarui. Dalam penelitian ini,

pengujian otomatis menggunakan *Vitest*.

## 11. Unified Modeling Language (UML)

*Unified Modeling Language* (UML) merupakan bahasa pemodelan visual yang menyediakan notasi untuk menggambarkan dan mendokumentasikan struktur serta perilaku sistem perangkat lunak. UML membantu menyajikan kebutuhan dan rancangan sistem secara terstruktur sehingga dapat dipahami oleh pengembang maupun pemangku kepentingan. UML berfungsi sebagai bahasa pemodelan, bukan sebagai tahapan atau metodologi pengembangan perangkat lunak tertentu (Larman, 2005).

Diagram UML yang digunakan untuk memodelkan sistem dapat menggambarkan sudut pandang yang berbeda. Diagram *use case* menunjukkan fungsi sistem yang berinteraksi dengan aktor, diagram aktivitas (*activity*) menggambarkan alur kerja, diagram urutan (*sequence*) menunjukkan pertukaran pesan menurut urutan waktu, sedangkan diagram kelas (*class*) menunjukkan struktur kelas dan hubungan antarkelas. Simbol, nama, dan fungsi notasi pada diagram tersebut dirangkum pada Tabel 2.1.

**Tabel 2.1:** Simbol, Nama, dan Fungsi Notasi UML

| **Simbol** | **Nama simbol** | **Fungsi simbol** |
|---|---|---|
|  | Aktor | Mewakili peran pengguna atau sistem eksternal yang berinteraksi dengan sistem; aktor berada di luar batas sistem dan dapat memulai atau berpartisipasi dalam suatu fungsi. |
|  | *Use case* | Menyatakan layanan atau tujuan yang disediakan sistem bagi aktor. Nama di dalam elips biasanya ditulis sebagai frasa kerja, misalnya “Mengunggah audio”. |
| Berlanjut ke halaman berikutnya | | |

**Tabel 2.1 – Lanjutan dari halaman sebelumnya**

| Simbol | Nama simbol | Fungsi simbol |
|---|---|---|
|  | Batas sistem | Persegi panjang yang membatasi ruang lingkup sistem; nama sistem dicantumkan pada bagian atas dan *use case* diletakkan di dalamnya, sedangkan aktor berada di luarnya. |
|  | Asosiasi | Garis penuh yang menghubungkan aktor dengan *use case* untuk menunjukkan partisipasi atau komunikasi. Pada diagram kelas, asosiasi yang sama menghubungkan kelas-kelas yang memiliki hubungan struktural. |
|  | Relasi *include* | Garis putus-putus berpanah dengan stereotipe «include» dari *use case* asal menuju *use case* yang disertakan. Menunjukkan bahwa perilaku *use case* tujuan selalu dipanggil sebagai bagian dari *use case* asal. |
|  | Relasi *extend* | Garis putus-putus berpanah dengan stereotipe «extend» dari *use case* tambahan menuju *use case* dasar. Menunjukkan perilaku tambahan yang dijalankan hanya jika kondisi atau titik ekstensi tertentu terpenuhi. |
|  | Simpul awal (*initial node*) | Lingkaran hitam penuh yang menandai titik awal aliran aktivitas; umumnya hanya ada satu simpul awal untuk satu alur proses. |

Berlanjut ke halaman berikutnya

**Tabel 2.1 – Lanjutan dari halaman sebelumnya**

| Simbol | Nama simbol | Fungsi simbol |
|---|---|---|
|  | Aksi (*action*) | Persegi panjang bersudut tumpul yang menyatakan satu langkah kerja atau aktivitas dalam proses; uraian aksi ditulis di dalam simbol. |
|  | Simpul keputusan atau penggabungan | Bentuk wajik yang digunakan untuk percabangan berdasarkan kondisi (dengan beberapa aliran keluar) atau menggabungkan aliran alternatif (dengan beberapa aliran masuk). Kondisi dapat ditulis sebagai *guard* pada aliran keluar. |
|  | Aliran kontrol (*control flow*) | Garis penuh berpanah yang menunjukkan urutan perpindahan kendali dari simpul aktivitas sebelumnya ke aksi atau simpul berikutnya. |
|  | Simpul *fork*/*join* | Batang tebal untuk memecah satu aliran menjadi beberapa aktivitas paralel (*fork*) atau menyinkronkan beberapa aliran paralel sebelum proses berlanjut (*join*). |
|  | Simpul akhir aktivitas | Lingkaran berbingkai dengan titik hitam di tengah yang menandai berakhirnya seluruh aktivitas pada alur tersebut. Berbeda dari simpul akhir aliran, simpul ini menghentikan aktivitas secara keseluruhan. |

Berlanjut ke halaman berikutnya

**Tabel 2.1 – Lanjutan dari halaman sebelumnya**

| Simbol | Nama simbol | Fungsi simbol |
|---|---|---|
|  | Partisipan (*participant*) | Persegi panjang di bagian atas diagram urutan yang berisi nama aktor atau objek yang terlibat dalam interaksi; garis hidup partisipan dimulai dari simbol ini. |
|  | Garis hidup (*lifeline*) | Garis vertikal putus-putus di bawah partisipan yang menunjukkan keberadaan partisipan sepanjang waktu interaksi; waktu dibaca dari atas ke bawah. |
|  | Aktivasi (*activation*) | Persegi panjang tipis pada garis hidup yang menunjukkan rentang waktu ketika partisipan sedang menjalankan operasi atau memiliki kendali. |
|  | Pesan sinkron | Garis penuh berujung panah penuh yang menunjukkan pemanggilan operasi; pengirim menunggu hingga penerima menyelesaikan operasi dan mengembalikan kendali. |
|  | Pesan balasan (*return message*) | Garis putus-putus berpanah yang menunjukkan kembalinya kendali atau hasil operasi dari penerima kepada pengirim pesan. |
|  | Kelas (*class*) | Persegi panjang yang dibagi menjadi nama kelas, atribut, dan operasi. Bagian atribut dan operasi dapat dikosongkan jika tidak diperlukan dalam model. |

Berlanjut ke halaman berikutnya

**Tabel 2.1 – Lanjutan dari halaman sebelumnya**

| Simbol | Nama simbol | Fungsi simbol |
|---|---|---|
|  | Asosiasi antarkelas | Garis penuh yang menunjukkan hubungan struktural antarkelas. Nama relasi, arah navigasi, dan multiplisitas pada ujung garis dapat ditambahkan untuk merinci hubungan. |
|  | Agregasi | Garis asosiasi dengan wajik kosong di sisi keseluruhan; menyatakan hubungan keseluruhan-bagian yang lemah, sehingga bagian tetap dapat berdiri sendiri ketika keseluruhan dihapus. |
|  | Komposisi | Garis asosiasi dengan wajik penuh di sisi pemilik; menyatakan hubungan keseluruhan-bagian yang kuat, dengan siklus hidup bagian bergantung pada keseluruhan. |
|  | Generalisasi | Garis penuh dengan segitiga kosong yang menunjuk ke kelas induk; menunjukkan pewarisan, sehingga kelas khusus mewarisi karakteristik kelas umum. |
|  | Dependensi | Garis putus-putus berpanah terbuka dari kelas yang bergantung menuju kelas yang menjadi pemasok; perubahan pada pemasok dapat memengaruhi kelas yang bergantung. |

*Sumber:* Diadaptasi dari Larman (2005).

## B. Literatur Review

Tinjauan pustaka dilakukan dengan menganalisis berbagai penelitian terdahulu yang memiliki relevansi dengan pemrosesan bahasa alami dan arsitektur

sistem terdistribusi. Data literatur tersebut dirangkum dalam Tabel 2.2 untuk memberikan gambaran komprehensif mengenai metode, tujuan, dan temuan utama dari setiap penelitian yang dirujuk.

**Tabel 2.2:** Ringkasan Literatur Review

| **Judul** | **Penulis** | **Metode Penelitian** | **Tujuan Penelitian** | **Hasil Penelitian** |
|---|---|---|---|---|
| User-centered Design Methods in Data-Intensive Software... | Riebel dkk. (2023) | *Literature Review* | Menganalisis integrasi UCD dalam AI | UCD meningkatkan empati dan definisi masalah dalam pengembangan AI. |
| Benchmarking open source and paid services for speech to text... | Ferraro dkk. (2023) | *Benchmark & WER Analysis* | Membandingkan performa layanan ASR berbayar dan open-source. | Layanan berbayar lebih akurat, namun performa sangat bergantung pada variasi input teks. |
| Evaluating Open-Source ASR Systems... | Imai dkk. (2025) | Evaluasi Eksperimental | Mengukur akurasi ASR pada audio bising | Perlu *robust error correction* untuk mengatasi bias demografis. |
| End-to-End Speech Recognition: A Survey | Prabhavalkar dkk. (2023) | *Survey* Literatur | Memetakan perkembangan E2E ASR | Model E2E mereduksi WER hingga 50% dibanding HMM tradisional. |
| Transformers in Speech Processing: A Survey | Latif dkk. (2025) | *Survey* Literatur | Menganalisis *Transformer* dalam audio | *Transformer* unggul dalam memodelkan dependensi jangka panjang. |
| Berlanjut ke halaman berikutnya | | | | |

**Tabel 2.2 – Lanjutan dari halaman sebelumnya**

| **Judul** | **Penulis** | **Metode Penelitian** | **Tujuan Penelitian** | **Hasil Penelitian** |
|---|---|---|---|---|
| Multilingual DistilWhisper... | Ferraz dkk. (2024) | Eksperimental (*Distillation*) | Meringankan model Whisper untuk bahasa langka | *DistilWhisper* meningkatkan performa bahasa *under-represented* dengan *overhead* minim. |
| Does ChatGPT and Whisper Make Humanoid Robots More Relatable? | Chen dkk. (2024) | Evaluasi Pengguna | Membandingkan Whisper vs Google ASR pada robot | Whisper memiliki WER terendah (1.71%) dan paling disukai oleh pengguna. |
| Quantization for OpenAI's Whisper Models... | Andreyev (2025) | Analisis Komparatif | Mengukur dampak kuantisasi pada latensi | Kuantisasi INT8 mereduksi size 45% dan latensi 19% tanpa degradasi akurasi. |
| A Self-Training Approach for Whisper to Enhance Long Dysarthric Speech... | Wang dkk. (2025) | *Self-training method* | Meningkatkan ASR untuk penderita disartria | Metode *self-training* meningkatkan performa pada ucapan penderita disabilitas. |
| Berlanjut ke halaman berikutnya | | | | |

**Tabel 2.2 – Lanjutan dari halaman sebelumnya**

| **Judul** | **Penulis** | **Metode Penelitian** | **Tujuan Penelitian** | **Hasil Penelitian** |
|---|---|---|---|---|
| Accurate Speech Recognition AI using Model of Deep Learning... | Isyanto dkk. (2024) | Eksperimental CNN | Membangun ASR untuk keamanan perbankan | Algoritma CNN mencapai akurasi 99% untuk pengenalan kata kunci keamanan. |
| Evaluating OpenAI's Whisper ASR... | Graham and Roll (2024) | Analisis Performa | Mengevaluasi Whisper pada berbagai aksen | Whisper lebih akurat pada aksen asli (*native*) dibandingkan aksen *non-native*. |
| The Role of TypeScript in Enhancing Development... | Sekhar Emmann (2021) | *Empirical Analysis* | Menganalisis peran TypeScript dalam pengembangan aplikasi *web* modern. | TypeScript secara signifikan mengurangi kesalahan *runtime* dan meningkatkan skalabilitas. |
| A Comparative Study of Agile and Waterfall Software Development... | Khan dan Mahadik (2024) | Analisis Komparatif | Membandingkan *Agile* vs *Waterfall* | *Agile* lebih unggul dalam kepuasan pelanggan dan kecepatan pengembangan. |

Berlanjut ke halaman berikutnya

**Tabel 2.2 – Lanjutan dari halaman sebelumnya**

| **Judul** | **Penulis** | **Metode Penelitian** | **Tujuan Penelitian** | **Hasil Penelitian** |
|---|---|---|---|---|
| Turning Whisper into Real-Time Transcription System | Macháček dkk. (2023) | Implementasi | Mengembangkan sistem transkripsi *real-time* berbasis Whisper. | *Whisper-Streaming* mencapai latensi 3,3 detik dengan akurasi yang tetap terjaga. |
| WhisperX: Time-Accurate Speech Transcription... | Bain dkk. (2023) | Pengembangan Sistem | Penyelarasan waktu tingkat kata pada Whisper | VAD dan *forced alignment* memungkinkan transkripsi 12x lebih cepat. |
| Bridging the Gap Between Generalized ASR and Domain-Specific... | Nelson dkk. (2025) | Integrasi Vosk | Menyesuaikan ASR untuk domain spesifik | Language modeling pada Vosk efektif untuk istilah medis dan hukum. |
| A semi-automatic pipeline for transcribing child speech | Christodoulidou dkk. (2025) | Evaluasi *Pipeline* | Membangun *pipeline* transkripsi suara anak | Koreksi manual sebelum *forced alignment* meningkatkan kualitas data akustik. |
| Comparing ASR Systems in Context of Speech Disfluencies | Teleki dkk. (2024) | Analisis Korpus Besar | Mengevaluasi Google ASR vs Whisper | Whisper lebih unggul dalam menangani interjeksi dan keraguan bicara. |

Berlanjut ke halaman berikutnya

**Tabel 2.2 – Lanjutan dari halaman sebelumnya**

| **Judul** | **Penulis** | **Metode Penelitian** | **Tujuan Penelitian** | **Hasil Penelitian** |
|---|---|---|---|---|
| Perancangan Antarmuka Pengguna Aplikasi Informasi Siswa... | Pratiwi dkk. (2022) | *User Centered Design* (UCD) | Merancang aplikasi informasi sekolah | Metode UCD menghasilkan desain yang 100% dapat digunakan oleh target *user*. |
| Estonian Speech Recognition and Transcription Editing Service | Olev and Alumäe (2022) | Pengembangan Layanan | Membangun sistem transkripsi berbasis wav2vec2.0 | Sistem berbasis E2E mencapai WER 6.9% dan mendukung *editing* kolaboratif. |

Berdasarkan tinjauan pustaka, penelitian ini menggabungkan evaluasi komparatif model *Whisper* pada korpus percakapan bahasa Indonesia dengan pengujian prototipe sistem transkripsi berbasis web. Prototipe memisahkan aplikasi *Next.js* dari unit inferensi *Python Worker* melalui antrean *Redis BullMQ*; evaluasi inferensi dilakukan pada GPU *Runpod*. RAD digunakan untuk menata proses penyempurnaan dan pengujian prototipe yang telah tersedia, sedangkan kegunaan antarmuka diukur melalui penyelesaian tugas dan SUS tanpa mengklaim siklus UCD penuh. Dengan demikian, fokus penelitian adalah kinerja model pada korpus yang ditetapkan serta evaluasi sistem sebagai alur kerja transkripsi dan koreksi.

# BAB III
# METODE PENELITIAN

## A. Jenis Penelitian

Penelitian ini merupakan penelitian rekayasa perangkat lunak dengan pendekatan *Research and Development* (R&D). Model *Rapid Application Development* (RAD) digunakan untuk menata penyempurnaan dan pengujian prototipe Skriptor. Prototipe awal telah tersedia sebelum evaluasi penelitian; kondisi awal, kebutuhan yang dievaluasi, dan penyempurnaan yang dilakukan didokumentasikan berdasarkan artefak aplikasi. Karena itu, penelitian ini tidak menyatakan telah menjalankan iterasi RAD atau UCD sebelumnya tanpa bukti yang mendukungnya. Kegunaan antarmuka dievaluasi melalui skenario tugas dan kuesioner SUS, bukan melalui klaim penerapan penuh siklus UCD.

Evaluasi penelitian mencakup tiga aspek. Pertama, pengujian fungsional melalui *black-box testing* manual dan pengujian unit otomatis menggunakan *Vitest*. Kedua, eksperimen komparatif tujuh varian *Whisper* yang mengukur WER, CER, waktu inferensi, RTF, dan penggunaan VRAM. Ketiga, evaluasi kegunaan antarmuka melalui penyelesaian tugas dan SUS.

## B. Setting Penelitian

Pengembangan aplikasi dan layanan pendukung dilakukan pada perangkat lokal, sedangkan eksperimen inferensi penelitian dijalankan pada *remote pod* Runpod dengan GPU NVIDIA RTX A5000 dan VRAM 24 GB karena perangkat lokal yang tersedia tidak memiliki sumber daya GPU yang memadai untuk menjalankan pengujian tersebut. Secara rancangan, aplikasi dan unit inferensi dapat ditempatkan pada lingkungan lokal apabila perangkat keras serta konfigurasi layanan yang dibutuhkan tersedia; deployment lokal bukan lingkungan yang dievaluasi dalam penelitian ini. Penelitian direncanakan berlangsung selama delapan bulan, mulai Desember 2026 hingga Juli 2027.

## C. Prosedur Penelitian

Prosedur penelitian mengikuti kerangka RAD dengan memperhatikan bahwa prototipe awal telah tersedia. Kegiatan pengembangan yang telah berlangsung dideskripsikan berdasarkan kode sumber dan artefak yang dapat diperiksa, sedangkan kegiatan yang belum dilakukan dinyatakan sebagai rencana. Tahap-tahap penelitian adalah sebagai berikut.

Pada tahap perencanaan kebutuhan (*requirements planning*), ditetapkan tujuan evaluasi, skenario fungsional, kebutuhan pengguna yang akan diuji, spesifikasi eksperimen, dan kriteria kelengkapan data. Arsitektur sistem yang tersedia—aplikasi *Next.js*, antrean *Redis BullMQ*, worker *Python*/*WhisperX*, penyimpanan, dan pelaporan progres—didokumentasikan berdasarkan implementasi.

Pada tahap desain pengguna (*user design*), alur utama penggunaan prototipe dan skenario tugas evaluasi ditetapkan. Tahap ini tidak mencakup penggalian kebutuhan atau iterasi desain bersama pengguna yang tidak benar-benar dilakukan. Skenario uji mencakup pengunggahan audio, pemantauan progres, navigasi dari segmen ke audio, koreksi teks, penyimpanan perubahan, dan ekspor hasil.

Pada tahap konstruksi (*construction*), prototipe yang sudah tersedia diuji secara fungsional dan disempurnakan apabila ditemukan ketidaksesuaian yang menghambat skenario penelitian. Implementasi sistem menggunakan antrean tugas untuk memisahkan permintaan web dari inferensi, sementara worker menjalankan *WhisperX* untuk transkripsi, penyelarasan waktu, dan, bila diaktifkan, diarization. Setiap perubahan yang dilakukan selama penelitian dicatat agar ruang lingkup penyempurnaan dapat dibedakan dari fitur yang telah tersedia sebelumnya.

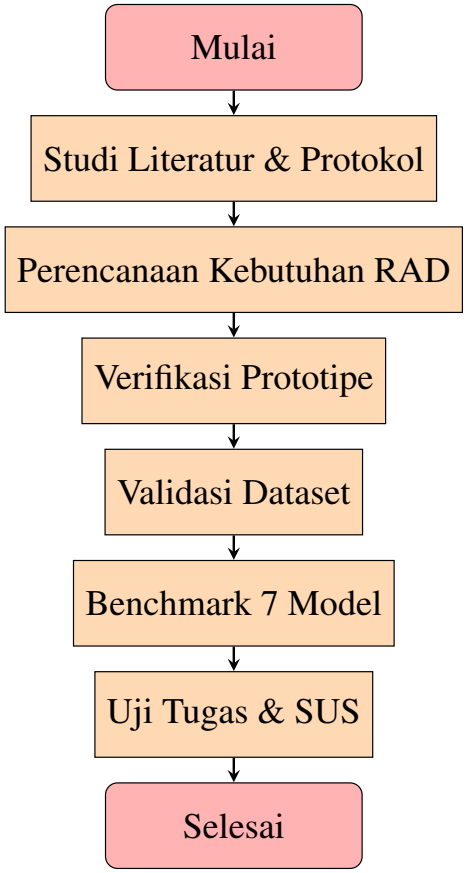
Pada tahap penerapan dan pengujian (*cutover*), prototipe yang telah diverifikasi digunakan dalam pengujian fungsional dan evaluasi kegunaan. Benchmark model dilakukan terpisah dari pilihan model operasional pada aplikasi: eksperimen mencakup *tiny, base, small, medium, large-v2, large-v3,* dan *turbo*, sedangkan antarmuka aplikasi menyediakan *small*, *turbo*, dan *large*. Hasil benchmark tidak dengan sendirinya mengubah pilihan model aplikasi.

Data utama terdiri atas seluruh 58 berkas audio utuh dari korpus *Mozilla Data Collective* (MDC): 16 episode *Homostoria* dan 42 episode *Hari Minggoean*. Berkas *TSV* acuan memuat 11.540 segmen transkrip beserta anotasi waktu. Sebelum eksperimen, kelengkapan berkas audio, kecocokan identitas episode, dan kesesuaian transkrip acuan diperiksa. Berkas hasil eksperimen pendahuluan yang hanya mencakup sebagian episode diperlakukan sebagai data pendahuluan dan tidak digabungkan dengan hasil evaluasi utama.

Pengujian fungsional dilakukan melalui *black-box testing* manual dan pengujian unit menggunakan *Vitest*. Skenario meliputi pengunggahan audio, penolakan format yang tidak didukung, pemantauan progres, pemutaran audio, koreksi dan penyimpanan segmen, serta ekspor transkrip. Eksperimen utama menjalankan masing-masing dari tujuh varian model pada semua 58 episode dengan konfigurasi perangkat lunak dan GPU yang sama. Setelah pemanasan setiap model menggunakan berkas uji yang tidak masuk perhitungan, waktu inferensi dan penggunaan VRAM setiap kombinasi model-episode diukur dalam tiga pengulangan. Median waktu dari tiga pengulangan digunakan sebagai waktu representatif untuk setiap kombinasi; VRAM puncak dicatat pada setiap pengulangan. Transkrip keluaran yang belum dikoreksi manusia dibandingkan dengan acuan setelah normalisasi seragam menggunakan WER dan CER. WER dan CER agregat dihitung dari jumlah operasi suntingan dan jumlah kata atau karakter acuan pada seluruh episode, bukan dari rerata sederhana antarepisode. Hasil pendahuluan yang tersedia untuk 55 berkas per model menjadi bahan pemeriksaan awal, bukan pengganti pengujian utama seluruh 58 berkas.

Evaluasi kegunaan dilakukan terhadap 30 responden yang dipilih secara *purposive sampling* dari kelompok mahasiswa, akademisi, dan pembuat konten yang memiliki kebutuhan transkripsi. Setiap responden menjalankan skenario tugas yang sama pada prototipe Skriptor, meliputi mengunggah audio, memahami progres, menemukan segmen melalui audio, melakukan koreksi, dan menyimpan hasil. Setelah tugas selesai, responden mengisi sepuluh butir SUS berskala Likert 5 poin. Keberhasilan penyelesaian tugas dan kendala yang diamati dicatat secara

deskriptif; skor SUS dihitung dan ditafsirkan berdasarkan acuan (Brooke, 1986; Bangor dkk., 2008). Uji tugas dan SUS merupakan evaluasi kegunaan, bukan bukti bahwa seluruh siklus UCD telah dilaksanakan.




**Gambar 3.1:** Alur Penelitian (*Flowchart*)

## D. Data dan Sumber Data

Bahan penelitian utama berupa data audio dan transkrip referensi (*ground truth*). Data penelitian menggunakan korpus *Mozilla Data Collective* (MDC) yang terdiri atas 16 episode *Homostoria* dan 42 episode *Hari Minggoean* dalam bahasa Indonesia. Dengan demikian, data berjumlah 58 berkas audio utuh per episode. Rekaman tersebut merupakan percakapan alami dengan total durasi 23,5 jam dan memiliki variasi intonasi, logat lokal, serta gangguan latar belakang (*noise*).

Berkas *TSV* (*tab-separated values*) digunakan sebagai *ground truth*. Berkas ini memuat 11.540 segmen transkrip manual beserta anotasi waktu dan berfungsi sebagai acuan untuk menghitung kesalahan pada tingkat kata maupun karakter.

Rincian statistik dataset disajikan pada Tabel 3.1.

**Tabel 3.1:** Statistik Dataset Penelitian

| **Nama Siniar** | **Jumlah Berkas** | **Jumlah Segmen** | **Total Durasi (Menit)** |
|---|---|---|---|
| Homostoria | 16 | 5.163 | 678,02 |
| Hari Minggoean | 42 | 6.377 | 735,43 |
| **Total** | **58** | **11.540** | **1.413,45** |

## E. Alat dan Instrumen Penelitian

Instrumen penelitian terdiri dari perangkat keras dan perangkat lunak yang dipilih untuk mendukung arsitektur sistem terdistribusi berperforma tinggi. Pemilihan komponen ini didasarkan pada kebutuhan skalabilitas (*scalability*) dan efisiensi dalam menangani model AI yang berat. Perangkat keras penelitian meliputi GPU NVIDIA RTX A5000 dengan VRAM 24 GB melalui *Runpod* untuk eksperimen inferensi, *Thin Client HP t630* dengan RAM 8 GB sebagai server aplikasi lokal, serta laptop *Lenovo IdeaPad Slim 3 14ALC6* dengan prosesor *AMD Ryzen 5 5500U*, RAM 20 GB, dan penyimpanan *NVMe SSD* untuk pengembangan serta pengujian lokal. Perangkat lokal tidak digunakan untuk benchmark inferensi model karena sumber daya GPU yang tersedia tidak memadai.

Perangkat lunak yang digunakan meliputi *Visual Studio Code*, *Git*, dan *Bun Runtime*. Lapisan aplikasi dibangun dengan *TypeScript*, *Next.js* 16, *React* 19, *Tailwind CSS* v4, *shadcn/ui*, dan *Drizzle ORM*. Infrastruktur data menggunakan *PostgreSQL*, *Redis* dengan *BullMQ*, serta *Garage S3*. Pemrosesan audio menggunakan *PyTorch*, *WhisperX*, *Faster-Whisper*, *CTranslate2*, *pyannote.audio* 3.1, dan *jiwer*, sedangkan pengujian unit dilakukan dengan *Vitest*.

Instrumen evaluasi yang digunakan adalah kuesioner *System Usability Scale* (SUS), yang terdiri atas sepuluh pernyataan berskala Likert 5 poin (Brooke, 1986). Kuesioner dibagikan setelah responden menyelesaikan skenario tugas pada antarmuka Skriptor. Responden berjumlah 30 orang dan dipilih secara *purposive sampling* dari kelompok mahasiswa, akademisi, serta pembuat konten digital yang memiliki kebutuhan transkripsi.

## F. Teknik Analisis Data

Analisis data dalam penelitian ini dilakukan melalui lima pendekatan yang saling melengkapi guna mengevaluasi aspek akurasi, efisiensi, kegunaan, dan fungsionalitas sistem secara komprehensif.

### 1. Pra-pemrosesan dan Normalisasi Teks

Sebelum dilakukan perbandingan antara teks hipotesis (keluaran ASR) dan teks referensi (*ground truth*), kedua teks terlebih dahulu melewati tahap normalisasi. Proses normalisasi meliputi pengubahan seluruh karakter ke huruf kecil (*case-folding*), penghapusan tanda baca menggunakan ekspresi reguler, dan penyeragaman spasi. Secara formal, fungsi normalisasi didefinisikan sebagai berikut:

$$f(\text{teks}) = \text{lowercase}(\text{strip}(\text{remove_punctuation}(\text{teks}))) \tag{3.1}$$

Teks yang telah dinormalisasi kemudian digunakan sebagai masukan untuk penghitungan metrik akurasi.

### 2. Analisis Akurasi ASR (WER dan CER)

Akurasi transkripsi diukur menggunakan dua metrik utama yang berbasis jarak *Levenshtein*. *Word Error Rate* (WER) mengukur tingkat kesalahan pada level kata dengan persamaan:

$$\text{WER} = \frac{S+D+I}{N} = \frac{S+D+I}{S+D+C} \tag{3.2}$$

dengan $S$ = jumlah substitusi, $D$ = jumlah penghapusan (*deletions*), $I$ = jumlah penyisipan (*insertions*), $C$ = jumlah kata benar (*correct*), dan $N$ = total kata pada teks referensi. *Character Error Rate* (CER) mengukur tingkat kesalahan pada level karakter menggunakan jarak *Levenshtein*:

$$\text{CER} = \frac{S_c + D_c + I_c}{N_c} \tag{3.3}$$

dengan $S_c$, $D_c$, dan $I_c$ berturut-turut adalah jumlah substitusi, penghapusan, dan penyisipan karakter, sedangkan $N_c$ adalah jumlah karakter pada teks referensi. Perhitungan WER dan CER dilakukan menggunakan pustaka *jiwer* pada *Python*. Nilai yang lebih rendah menunjukkan akurasi transkripsi yang lebih baik.

### 3. Analisis Efisiensi Komputasi

Efisiensi komputasi diukur melalui *Real-Time Factor* (RTF) dan penggunaan VRAM puncak. RTF merupakan rasio antara waktu inferensi dan durasi audio yang diproses:

$$\text{RTF} = \frac{\Delta t_{\text{inferensi}}}{\Delta t_{\text{audio}}} \tag{3.4}$$

Nilai RTF $< 1$ menunjukkan bahwa sistem memproses audio lebih cepat dari waktu nyata. Kecepatan pemrosesan juga dapat dinyatakan sebagai *Speed Factor* (Speed $= 1/\text{RTF}$). Penggunaan VRAM puncak (*Peak VRAM*) dicatat melalui *polling* perintah `nvidia-smi` atau fungsi `torch.cuda.memory_reserved()` pada *PyTorch*. Data tersebut digunakan untuk menentukan kelayakan model pada berbagai spesifikasi GPU.

### 4. Analisis Kegunaan (*Usability*) dengan SUS

Evaluasi kegunaan antarmuka dilakukan menggunakan kuesioner *System Usability Scale* (SUS) yang terdiri dari 10 butir pernyataan berskala Likert 1–5 (Brooke, 1986). Kuesioner didistribusikan kepada 30 responden. Skor SUS untuk setiap responden dihitung dengan formula:

$$\text{SUS} = 2{,}5 \times \left[ \sum_{i \in \text{ganjil}} (x_i - 1) + \sum_{j \in \text{genap}} (5 - x_j) \right] \tag{3.5}$$

dengan $x_i$ adalah skor butir ganjil (pernyataan positif) dan $x_j$ adalah skor butir genap (pernyataan negatif). Rentang skor SUS adalah 0–100. Rata-rata dan standar deviasi skor SUS kemudian dihitung dan diinterpretasikan berdasarkan skala penerimaan (*Acceptability Ranges*), skala huruf (*Grade Scale*), dan peringkat kata sifat (*Adjective*

*Rating*) dari Bangor dkk. (2008).

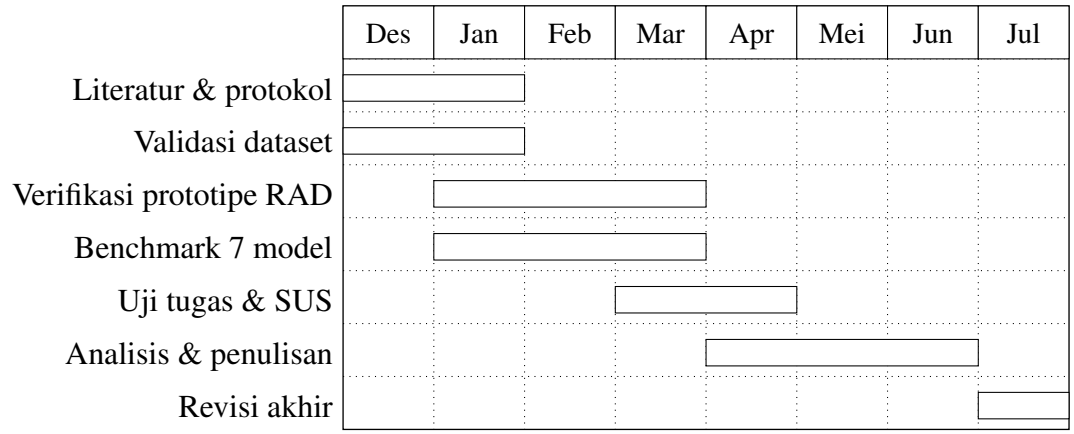
### 5. Analisis Pengujian Fungsional

Pengujian fungsional dilakukan melalui pendekatan kotak hitam manual (*manual black-box testing*). Setiap fitur utama sistem diuji berdasarkan matriks skenario pengujian yang memuat: identifikasi skenario, kondisi masukan (*input*), langkah pengujian, respons sistem yang diharapkan (*expected result*), dan hasil pengamatan aktual (*actual result*). Setiap skenario dinilai dengan status "Berhasil" atau "Gagal" berdasarkan kesesuaian antara keluaran aktual dan keluaran yang diharapkan.

## G. Jadwal Penelitian

Penelitian direncanakan berlangsung selama delapan bulan, mulai Desember 2026 hingga Juli 2027. Rincian kegiatan disajikan dalam bentuk *Gantt Chart* pada Tabel 3.2.

**Tabel 3.2:** Jadwal Penelitian (*Gantt Chart*)




*Keterangan:* Desember pada M1 adalah Desember 2026; Januari–Juli adalah tahun 2027.

# BAB IV
# PENUTUP

## A. Kesimpulan

Proposal ini merancang penelitian rekayasa perangkat lunak dengan pendekatan *Research and Development* (R&D) dan model *Rapid Application Development* (RAD) untuk menyempurnakan serta menguji prototipe sistem transkripsi Skriptor. Evaluasi yang direncanakan mencakup pengujian fungsional, perbandingan tujuh varian *Whisper* pada 58 episode percakapan berbahasa Indonesia menggunakan metrik WER, CER, waktu inferensi, RTF, dan penggunaan VRAM, serta pengujian tugas dan pengukuran SUS terhadap 30 responden.

Eksperimen inferensi direncanakan menggunakan GPU NVIDIA RTX A5000 melalui layanan *Runpod*; kinerja pada perangkat lokal tidak menjadi objek pengujian. Karena penelitian masih berada pada tahap proposal, uraian ini merangkum rancangan dan ruang lingkup penelitian, bukan menyatakan hasil atau menarik kesimpulan empiris.

## B. Saran

Sebelum eksperimen dilaksanakan, kelengkapan audio dan transkrip acuan perlu diverifikasi serta prosedur normalisasi teks, konfigurasi model, dan lingkungan komputasi ditetapkan secara konsisten. Setiap pengulangan, waktu inferensi, dan penggunaan VRAM perlu dicatat agar hasil perbandingan dapat ditelusuri dan direproduksi.

Pengujian fungsional dan evaluasi kegunaan perlu dilakukan sesuai skenario yang telah dirumuskan. Pelaksanaan uji kepada responden hendaknya disertai penjelasan dan persetujuan yang diperlukan, serta hasilnya dilaporkan berdasarkan data yang benar-benar diperoleh. Setelah seluruh pengujian selesai, kesimpulan akhir dan saran pengembangan perlu disesuaikan dengan temuan empiris, bukan hanya dengan target atau rencana penelitian.

## DAFTAR PUSTAKA


Andreyev, A. (2025). *Quantization for OpenAI's whisper models: A comparative analysis.* arXiv. Diakses pada 2026-04-14 http://arxiv.org/abs/2503.09905 doi: 10.48550/arXiv.2503.09905

Bain, M., Huh, J., Han, T., & Zisserman, A. (2023). WhisperX: Time-accurate speech transcription of long-form audio. In *INTERSPEECH 2023* (pp. 4489–4493). ISCA. https://doi.org/10.21437/Interspeech.2023-78

Brooke, J. (1986). SUS - a quick and dirty usability scale.

Chen, X., Luo, K., Gee, T., & Nejati, M. (n.d.). *Does ChatGPT and whisper make humanoid robots more relatable?* arXiv. Diakses pada 2026-04-14 http://arxiv.org/abs/2402.07095 doi: 10.48550/arXiv.2402.07095

Christodoulidou, P., Tanner, J., Stuart-Smith, J., McAuliffe, M., Murali, M., Smith, A., ... Kuschmann, A. (n.d.). A semi-automatic pipeline for transcribing and segmenting child speech. In *Interspeech 2025* (pp. 4278–4282). ISCA. Diakses pada 2026-04-14 https://www.isca-archive.org/interspeech_2025/christodoulidou25_interspeech.html doi: 10.21437/Interspeech.2025-1030

Fathony, I. A. N., Mareta, A., Adiana, B. E., Wardhani, O., & Halim, D. A. (2025). Optimisasi Whisper speech-to-text Bahasa Indonesia dengan hybrid cloud dan multi-engine. *Elkom: Jurnal Elektronika dan Komputer*, *18*(1), 60–72. https://doi.org/10.51903/5n2d3s08

Ferraro, A., Galli, A., La Gatta, V., & Postiglione, M. (2023). Benchmarking open source and paid services for speech to text: an analysis of quality and input variety. *Frontiers in Big Data*, *6*, 1210559. https://doi.org/10.3389/fdata.2023.1210559

Ferraz, T. P., Boito, M. Z., Brun, C., & Nikoulina, V. (n.d.). Multilingual DistilWhisper: Efficient distillation of multi-task speech models via language-specific experts. In *ICASSP 2024 - 2024 IEEE international conference on acoustics, speech and signal processing (ICASSP)* (pp. 10716–10720). Diakses pada 2026-04-14 http://arxiv.org/abs/2311.01070 doi: 10.1109/ICASSP48485.2024.10447520

Gaur, Y., Lasecki, W. S., Metze, F., & Bigham, J. P. (2016). The effects of automatic speech recognition quality on human transcription latency. In *Proceedings of the 13th international web for all conference* (pp. 1–8). ACM. https://doi.org/10.1145/2899475.2899478

Graham, C., & Roll, N. (n.d.). Evaluating OpenAI's whisper ASR: Performance analysis across diverse accents and speaker traits. , *4*(2), 025206. Diakses pada 2026-04-14 https://pubs.aip.org/jel/article/4/2/025206/3267247/Evaluating-OpenAI-s-Whisper-ASR-Performance doi: 10.1121/10.0024876

Imai, S., Chowdhury, T., & Stent, A. J. (2025). Evaluating open-source ASR systems: Performance across diverse audio conditions and error correction methods. In *Proceedings of the 31st international conference on computational linguistics* (pp. 5027–5039). Association for Computational Linguistics. https://

aclanthology.org/2025.coling-main.336/

Isyanto, H., Ibrahim, W., Samsinar, R., & Sudarwati, W. (n.d.). Accurate speech recognition AI using model of deep learning for security access.

Khan, S., & Mahadik, S. (n.d.). A comparative study of agile and waterfall software development methodologies. , 54–57. Diakses pada 2026-04-14 http://ijarsct.co.in/Paper15207.pdf doi: 10.48175/IJARSCT-15207

Larman, C. (2005). *Applying UML and patterns: an introduction to object-oriented analysis and design and iterative development* (3rd ed ed.). Prentice Hall PTR, c2005.

Latif, S., Zaidi, A., Cuayahuitl, H., Shamshad, F., Shoukat, M., Usama, M., & Qadir, J. (n.d.). *Transformers in speech processing: A survey.* arXiv. Diakses pada 2026-04-14 http://arxiv.org/abs/2303.11607 doi: 10.48550/arXiv.2303.11607

Macháček, D., Dabre, R., & Bojar, O. (2023). Turning whisper into real-time transcription system. In *Proceedings of the 13th international joint conference on natural language processing and the 3rd conference of the asia-pacific chapter of the association for computational linguistics: System demonstrations* (pp. 17–24). Association for Computational Linguistics. Diakses pada 2026-04-14 https://aclanthology.org/2023.ijcnlp-demo.3 doi: 10.18653/v1/2023.ijcnlp-demo.3

Martin, J. (1991). *Rapid application development*. Macmillan Publishing Company.

Nelson, J., & Gretkzi, M. (n.d.). Bridging the gap between generalized ASR and domain-specific communication using vosk-driven language modeling.

Olev, A., & Alumäe, T. (2022). Estonian speech recognition and transcription editing service. *Baltic Journal of Modern Computing*, *10*(3). https://doi.org/10.22364/bjmc.2022.10.3.14

Prabhavalkar, R., Hori, T., Sainath, T. N., Schlüter, R., & Watanabe, S. (2023). *End-to-end speech recognition: A survey.* arXiv. Diakses pada 2026-04-14 http://arxiv.org/abs/2303.03329 doi: 10.48550/arXiv.2303.03329

Pratiwi, A. S., Wijoyo, S. H., & Wicaksono, S. A. (n.d.). Perancangan antarmuka pengguna aplikasi sistem informasi siswa berbasis android menggunakan metode user centered design (studi pada : di SMK telkom banjarbaru).

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022). Robust speech recognition via large-scale weak supervision. *arXiv*. https://doi.org/10.48550/arXiv.2212.04356

Riebel, J. A. (2023). User-centered design methods in data-intensive software development processes: A state-of-the-art review.

Sekhar Emmanni, P. (n.d.). The role of TypeScript in enhancing development with modern JavaScript frameworks. , *10*(2), 1738–1741. Diakses pada 2026-04-14 https://www.ijsr.net/getabstract.php?paperid=SR24401234212 doi: 10.21275/SR24401234212

Teleki, M., Dong, X., Kim, S., & Caverlee, J. (2024). Comparing ASR systems in the context of speech disfluencies. In *Interspeech 2024* (pp. 4548–4552). ISCA. https://doi.org/10.21437/Interspeech.2024-1270

Wang, S., Zhou, J., Zhao, S., & Qin, Y. (n.d.). *A self-training approach for whisper to enhance long dysarthric speech recognition.* arXiv. Diakses pada 2026-04-14 http://arxiv.org/abs/2506.22810 doi: 10.48550/arXiv.2506.22810